

EL MECANISMO DE REACCIÓN

K. V Ramirez Ambrocio

7690-22-2239 Universidad Mariano Gálvez

Seminario de Tecnologías de Información

kramireza10@miumg.edu.gt

Resumen

Dentro del mismo, se tiene como objetivo identificar y explicar las buenas prácticas que debe seguir un desarrollador al construir aplicaciones reactivas, lo que se entiende por aplicaciones que responden de forma rápida, estable y sin bloquearse ante cargas de trabajo variables. Se obtuvieron conceptos de información aplicada donde consistió en leer artículos técnicos y académicos sobre programación reactiva, el manifiesto reactivo y la especificación de Reactive Streams, que se complementan con documentación oficial de frameworks utilizados en la industria como Project Reactor y Spring WebFlux. En estos resultado se identificaron cuatro buenas prácticas centrales: mantener los flujos de datos sin estado , manejar correctamente la contrapresión (backpressure), evitar operaciones bloqueantes dentro de los flujos reactivos, y liberar las suscripciones cuando ya no se necesitan. Se concluye que programar de forma reactiva no consiste únicamente en usar una librería específica, sino en adoptar un cambio de mentalidad: que se debe de diseñar pensando en que los datos llegan de forma asíncrona es decir no de manera paralela sino simultanea y que el sistema debe seguir respondiendo incluso cuando algo falla. Aplicar estas prácticas evita errores comunes como el mal uso de recursos y mejora la experiencia de quien usa la aplicación que al final es lo que deseamos para nuestra app o sistema.

Palabras clave: programación reactiva, contrapresión, aplicaciones asíncronas, resiliencia, Reactive Streams.

Desarrollo del tema

Para entender qué es una buena práctica en aplicaciones reactivas, primero hay que tener claro qué es una aplicación reactiva. En pocas palabras, es una aplicación diseñada para responder rápido y de forma estable ante eventos, sin importar si esos eventos llegan de a poco o llegan miles por segundo. Esta forma de programar se popularizó porque las aplicaciones actuales ya no manejan solo unos cuantos usuarios conectados al mismo tiempo, sino miles o millones, y el modelo tradicional de programación, donde el sistema espera a que una tarea termine antes de seguir con la siguiente, simplemente no aguanta esa exigencia.

La base viene del llamado Manifiesto Reactivo, un documento publicado por un grupo de ingenieros de software que establece que un sistema reactivo debe cumplir cuatro características: ser responsivo, es decir, responder en un tiempo razonable; ser resiliente, o sea, seguir funcionando aunque una parte falle; ser elástico, capaz de escalar hacia arriba o hacia abajo según la demanda; y estar orientado a mensajes, comunicándose de forma asíncrona entre sus componentes. Estas cuatro ideas no son un capricho técnico, sino la respuesta a un problema real: los sistemas modernos no pueden asumir que todo va a funcionar siempre bien ni que la carga de trabajo va a ser siempre la misma.

Se puede hablar de las buenas prácticas concretas que cualquier persona desarrolladora debería

aplicar. La primera es mantener los flujos de datos sin estado. Esto quiere decir que cada paso del flujo reactivo no debería depender de información guardada de pasos anteriores, porque eso complica probar el código y también dificulta que el sistema procese varias cosas en paralelo. Cuando un flujo es stateless, es mucho más fácil escalarlo, porque no importa en qué servidor o hilo se ejecute, siempre va a dar el mismo resultado con la misma entrada.

La segunda buena práctica, y probablemente la más importante, es manejar correctamente la contrapresión, conocida en inglés como backpressure. Este concepto resuelve un problema muy común: qué pasa cuando un productor de datos genera información más rápido de lo que el consumidor puede procesarla. Sin un mecanismo de contrapresión, esos datos se acumulan en memoria sin control, y tarde o temprano la aplicación se cae con un error de memoria insuficiente (tutorialQ, 2026). La especificación de Reactive Streams resuelve esto dejando que sea el consumidor quien le diga al productor cuántos elementos puede recibir en un momento dado, en lugar de que el productor simplemente le envíe todo de golpe. Es importante distinguir esto de otro concepto parecido, que es el límite de tasa o rate limiting: la contrapresión es adaptativa, porque cambia según la capacidad real del consumidor en cada momento, mientras que el límite de tasa es una regla fija, como "máximo cien solicitudes por segundo", sin importar si el consumidor podría procesar más o menos en ese instante.

La tercera práctica tiene que ver con evitar operaciones bloqueantes dentro de un flujo reactivo. Muchas veces el problema no está en el código reactivo en sí, sino en que dentro de ese flujo se llama a una operación tradicional que sí bloquea, como una consulta a una base de datos que no es reactiva. Cuando eso pasa, se pierde toda la ventaja de programar de forma asíncrona, porque ese punto del flujo va a quedar esperando igual que en el modelo tradicional. La recomendación de la literatura técnica es aislar esas operaciones bloqueantes en grupos de hilos separados, para que no afecten el resto del sistema reactivo.

La cuarta práctica, menos comentada pero igual de relevante, es liberar las suscripciones cuando ya no se necesitan. Una suscripción activa consume recursos como memoria e hilos de ejecución, y si no se cierra correctamente cuando ya cumplió su función, esos recursos quedan ocupados sin necesidad, lo cual con el tiempo puede degradar el rendimiento de toda la aplicación.

Gran parte del contenido disponible en línea sobre programación reactiva proviene de blogs técnicos y documentación de frameworks específicos, los cuales son útiles para resolver problemas puntuales, pero no siempre tienen el mismo respaldo que la literatura relacionada del tema. El Manifiesto Reactivo y el trabajo original de Elliott y Hudak (1997) sobre programación reactiva funcional sí constituyen fuentes primarias reconocidas académicamente, y son las que sustentan los conceptos que después la industria adaptó a lenguajes como Java, JavaScript o Kotlin.

Observaciones y comentarios

Al revisar este tema, algo que llama la atención es que el término reactivo "se usa de forma un poco suelta en la industria: muchas veces se dice que una aplicación es reactiva solo porque usa una librería como RxJS o Project Reactor, sin necesariamente cumplir con los cuatro principios del Manifiesto Reactivo. Esto puede generar una falsa sensación de que basta con cambiar de librería para tener un sistema resiliente y elástico, cuando en realidad esos atributos dependen de decisiones de arquitectura completas, no solo del código.

También se observa que la contrapresión, siendo el concepto más técnico y más importante, es el que menos se enseña de forma clara en tutoriales introductorios, que suelen enfocarse en la sintaxis de los operadores reactivos sin explicar por qué existen. Esto puede llevar a que una persona desarrolladora use estas herramientas sin entender completamente el problema que están resolviendo.

Como limitación de este trabajo, no se profundizó en comparaciones de rendimiento entre distintos frameworks reactivos, porque eso requeriría pruebas técnicas propias y no solo revisión bibliográfica. Como posible línea de investigación futura, sería interesante analizar cómo se aplican estos mismos principios reactivos en frameworks de frontend modernos, más allá del backend, donde también se están adoptando conceptos similares.

Conclusiones

1. Una aplicación reactiva no se define por la librería que usa, sino por cumplir los cuatro principios del Manifiesto Reactivo: ser responsiva, resiliente, elástica y orientada a mensajes.
2. Mantener los flujos de datos sin estado facilita las pruebas del sistema y permite que este escale sin depender de dónde se ejecute cada parte del código.
3. El manejo correcto de la contrapresión es la práctica más crítica, ya que su ausencia es la causa más común de fallas por saturación de memoria en sistemas con alto volumen de datos.
4. Evitar operaciones bloqueantes dentro de flujos reactivos, y aislarlas cuando sean necesarias, es indispensable para no perder los beneficios del modelo asíncrono.
5. Cerrar las suscripciones que ya no se usan es una práctica sencilla pero frecuentemente olvidada, y su descuido afecta directamente el rendimiento a largo plazo de la aplicación.

Referencias

- Baeldung. (2025, 26 de marzo). *Backpressure mechanism in Spring WebFlux*. <https://www.baeldung.com/spring-webflux-backpressure>
- Bonér, J., Farley, D., Kuhn, R., & Wampler, M. (2014). *The reactive manifesto* (v.2.0). <https://www.reactivemanifesto.org/>
- Elliott, C., & Hudak, P. (1997). Functional reactive animation. *Proceedings of the 1997 ACM SIGPLAN International Conference on Functional Programming*, 32(8), 263–273. <https://doi.org/10.1145/258948.258973>
- javaspring.net. (s.f.). *Reactive programming in Java: A comprehensive guide*. <https://www.javaspring.net/blog/reactive-programming-in-java/>
- tutorialQ. (2026). *Backpressure in reactive programming*. <https://tutorialq.com/dev/microservices/backpressure-in-reactive-programming>
- Repositorio Git: <https://github.com/Valkill/seminario-encuesta.git>